

Sistemas Embebidos I

Otoño 2026

Práctica #8: ADC

Ana

Camila

Hamid

Jonathan

Eduardo

Fecha:

25-10-2026

Índice

Práctica #8: ADC *2*
Introducción **2**
Descripción de la práctica **2**
Objetivo **2**
Marco teórico **2**
Desarrollo **3**
Materiales **3**
Esquemático **4**
Desarrollo **4**
Configuración e inicialización del sistema 4
Inicialización del ADC 4
Configuración de la señal PWM 5
Captura de muestras del ADC 5
Envío de datos a la consola 5
Procesamiento de comandos 6
Aplicación de filtros digitales 6
Control del LED mediante PWM 7
Ciclo principal del programa 7
Registro y procesamiento de datos 7
Resultados **8**
Análisis del ruido en las lecturas del ADC 8
Muestreo de una señal PWM 8
Comparación de la respuesta de los filtros digitales 8
Registro de datos del dimmer en la consola 9
Evidencias **10**
Conclusiones **16**

Práctica #8: ADC

Introducción

Descripción de la práctica

En esta práctica se trabaja con el convertidor analógico-digital (ADC) de la Raspberry Pi Pico 2, con el propósito de observar cómo se pueden leer señales analógicas y convertirlas en valores digitales que el microcontrolador pueda interpretar.

Durante la práctica se utilizan diferentes conexiones para medir señales, como la de un potenciómetro y una señal PWM. También se observa cómo cambian las lecturas dependiendo de la frecuencia de muestreo y cómo pueden presentarse variaciones debido al ruido. Además, se realizan pruebas para identificar el aliasing y se utilizan filtros digitales para mejorar las mediciones.

Finalmente, se busca relacionar las lecturas obtenidas con el control de una salida PWM, para comprender cómo se pueden utilizar los datos de entrada para controlar un dispositivo electrónico.

Objetivo

Comprender el funcionamiento del ADC de la Raspberry Pi Pico 2 mediante la lectura de señales analógicas, identificando cómo influyen el muestreo, la resolución, el ruido y el aliasing en las mediciones. También se busca aplicar filtros digitales para obtener lecturas más estables y utilizar los valores obtenidos para controlar una salida PWM.

Marco teórico

- 1. Convertidor analógico-digital (ADC): Es el componente que convierte una señal analógica, como el voltaje de un potenciómetro, en un valor digital que puede interpretar el microcontrolador.
- 2. Muestreo: Es el proceso de tomar lecturas de una señal en diferentes momentos. La frecuencia de muestreo indica cuántas muestras se toman por segundo.
- 3. Frecuencia de muestreo: Permite conocer la rapidez con la que se realizan las lecturas de una señal. Una frecuencia adecuada ayuda a representar correctamente sus cambios.

- 4. Aliasing: Es un fenómeno que ocurre cuando la frecuencia de muestreo es demasiado baja, provocando que una señal parezca tener un comportamiento diferente al real.
- 5. Regla de Nyquist: Establece que se deben tomar suficientes muestras para representar correctamente la frecuencia más alta de una señal y evitar errores de interpretación.
- 6. Cuantización: Es el proceso mediante el cual el ADC asigna un valor digital al voltaje medido, de acuerdo con los niveles que puede representar.
- 7. Resolución del ADC: Indica la cantidad de niveles digitales disponibles para representar una señal. En la Raspberry Pi Pico 2, el ADC tiene una resolución de 12 bits, lo que permite obtener valores desde 0 hasta 4095.
- 8. Error de cuantización: Es la diferencia entre el voltaje real de entrada y el valor digital que representa el ADC. Se produce porque el convertidor trabaja con niveles definidos.
- 9. Ruido: Son las variaciones que pueden aparecer en las lecturas debido a factores como la alimentación eléctrica, las conexiones y la actividad del microcontrolador.
- 10. Filtro anti-alias: Es un filtro que reduce las frecuencias altas antes de que la señal llegue al ADC, ayudando a evitar errores de interpretación.
- 11. Media móvil: Es un filtro digital que calcula el promedio de varias lecturas recientes para reducir las variaciones causadas por el ruido y obtener resultados más estables.
- 12. Media móvil exponencial (EMA): Es un método de filtrado que combina la lectura nueva con el promedio anterior. Permite suavizar las mediciones sin necesidad de almacenar todas las lecturas anteriores.
- 13. Señal PWM: Es una técnica que permite controlar una salida digital mediante el tiempo que permanece encendida y apagada. Se puede utilizar para regular el brillo de un LED u otras aplicaciones de control.
- 14. Canales ADC del RP2350: Son las entradas que permiten medir diferentes señales analógicas. En esta práctica se utilizan los pines GP26, GP27 y GP28, correspondientes a los canales ADC0, ADC1 y ADC2.
- Funciones del ADC en el SDK de C: Son instrucciones que permiten inicializar el convertidor, configurar los pines analógicos, seleccionar el canal de lectura y obtener los valores digitales de la señal.

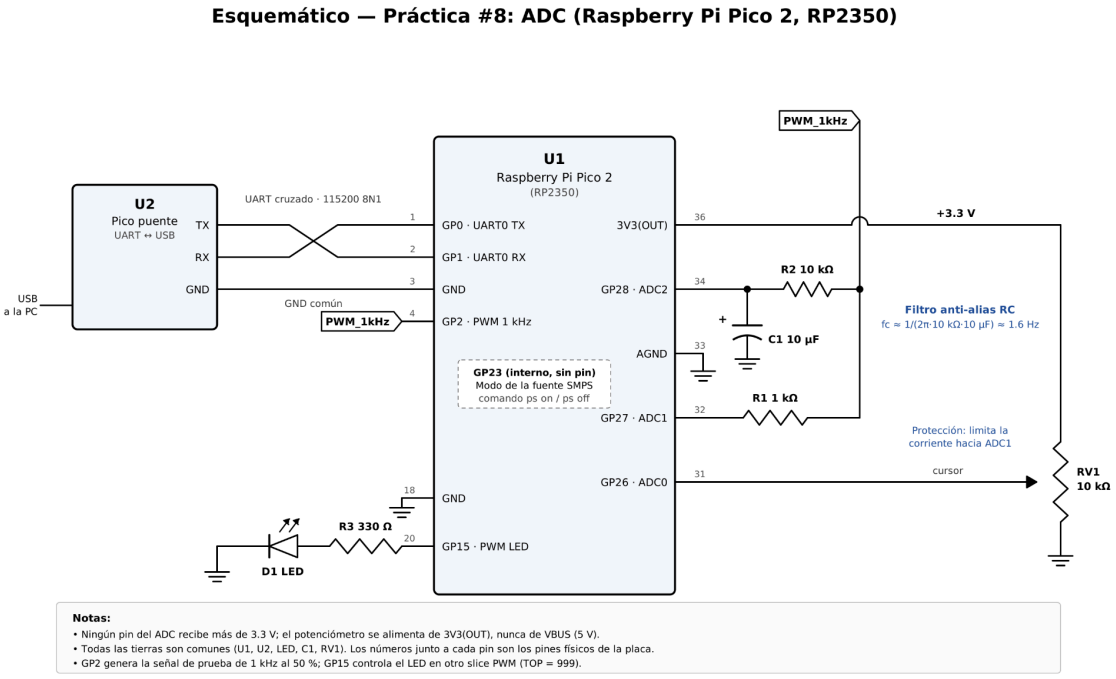
Desarrollo

Materiales

- Raspberry Pi Pico 2 (RP2350).
- Potenciómetro de 10 kΩ.
- Resistencia de 1 kΩ.

- Resistencia de 10 kΩ.
- Resistencia de 330 Ω.
- Capacitor de 10 µF.
- LED.
- Protoboard.
- Cables jumper para las conexiones.
- Computadora con el SDK de Raspberry Pi Pico y un entorno de desarrollo para programar en lenguaje C.
- Cable USB para conectar la Raspberry Pi Pico 2 a la computadora.
- Una segunda Raspberry Pi Pico para el puente de comunicación UART.
- Cables para realizar las conexiones de comunicación UART.

Esquemático



Desarrollo

Configuración e inicialización del sistema

Se utilizó la Raspberry Pi Pico 2, programada en lenguaje C mediante el SDK de Raspberry Pi Pico. Al inicio del programa se configuraron los periféricos necesarios para realizar las lecturas analógicas, generar señales PWM y establecer la comunicación con la consola.

Se utilizaron los pines GP26, GP27 y GP28 como entradas analógicas, correspondientes a los canales ADC0, ADC1 y ADC2. Además, se configuró GP2

para generar una señal PWM de prueba, GP15 para controlar el brillo de un LED y GP23 para modificar el estado de una salida digital.

Inicialización del ADC

Para preparar el convertidor analógico-digital se utilizaron las funciones de inicialización y configuración de los pines analógicos.

```
adc_init();
adc_gpio_init(26);
adc_gpio_init(27);
adc_gpio_init(28);
```

La función **adc_init()** inicializa el periférico ADC, mientras que **adc_gpio_init()** configura los pines que se utilizarán para las mediciones.

Posteriormente, mediante **adc_select_input()** se selecciona el canal que se desea leer y con **adc_read()** se obtiene el código digital correspondiente al voltaje presente en la entrada.

Configuración de la señal PWM

Se configuró GP2 para generar una señal PWM de prueba con una frecuencia de 1 kHz y un ciclo de trabajo aproximado del 50 %. Esta señal se utiliza como entrada para estudiar el muestreo y analizar la relación entre la frecuencia de la señal y la frecuencia con la que el ADC toma las lecturas.

Por otra parte, GP15 se configuró como salida PWM para controlar un LED. Para ello, se estableció un rango de 1000 niveles de control, permitiendo modificar el ciclo de trabajo de acuerdo con el valor procesado por el programa.

Captura de muestras del ADC

La adquisición de datos se realiza mediante las funciones del archivo **adc_stream.c**. La función **adc_stream_capturar()** recibe el canal que se desea medir, el número de muestras y la frecuencia de muestreo.

Antes de realizar la captura, se verifican los parámetros para evitar solicitudes fuera de los límites establecidos. Después, se selecciona el canal ADC, se determina el intervalo entre muestras y se almacenan las lecturas en un arreglo.

```
adc_select_input(canal);

for (uint32_t i = 0; i < n; i++) {
    while (time_us_32() - t < periodo_us) { }
    t += periodo_us;
    buf[i] = adc_read();
}
```

Este fragmento muestra la selección del canal y la adquisición periódica de las muestras. El arreglo **buf** almacena los códigos digitales para que puedan procesarse o enviarse a la consola.

Envío de datos a la consola

Una vez capturadas las muestras, la función **adc_stream_enviar()** se encarga de transmitir los datos mediante la comunicación serial.

```
for (uint32_t i = 0; i < n; i++) {
    consola_printf("%lu %u\n",
                   (unsigned long)i, buf[i]);
    consola_flush();
}
```

En este fragmento se envía el índice de cada muestra junto con su valor digital. Los datos pueden guardarse en un archivo CSV para su procesamiento posterior mediante herramientas de análisis y generación de gráficas.

Procesamiento de comandos

La función **procesar()** permite interpretar las instrucciones introducidas desde la consola, como la selección de la frecuencia de muestreo, el canal ADC, el tipo de filtro y el control del LED.

Entre los comandos contemplados en la práctica se encuentran **adc, fs, filtro, ps, stream y dimmer**. Cada uno está destinado a configurar una operación específica del sistema.

El comando **stream** permite solicitar una captura de muestras, mientras que los demás comandos permiten consultar mediciones o modificar los parámetros de funcionamiento. La ejecución de estas operaciones depende de la implementación de sus respectivos bloques de procesamiento.

Aplicación de filtros digitales

El programa contempla tres opciones de filtrado: sin filtro, media móvil y media móvil exponencial (EMA).

La media móvil utiliza las últimas 16 muestras para calcular un promedio, mientras que la EMA combina la lectura actual con el promedio anterior mediante un factor de ajuste de 1/16.

La función **filtrar()** está destinada a procesar las muestras de acuerdo con el método seleccionado. En la plantilla original, esta función aparece marcada como pendiente de implementación, por lo que es necesario completar sus operaciones para que realice el filtrado.

Control del LED mediante PWM

El control del brillo se plantea mediante la función **tarea_dimmer()**, que debe ejecutarse periódicamente para leer el potenciómetro, procesar su valor y actualizar el nivel de PWM del LED.

El procedimiento contempla la lectura del canal ADC0, la aplicación del filtro seleccionado y la conversión del código digital a un nivel de control entre 0 y 999. Posteriormente, la función **led_set_level()** actualiza la salida PWM de GP15.

También se contempla un contador que registra el número de cambios del ciclo de trabajo durante un segundo. En la plantilla original, la función **tarea_dimmer()** aparece pendiente de implementación.

Ciclo principal del programa

La función **main()** se encarga de inicializar la comunicación UART, preparar las entradas analógicas, configurar las salidas digitales y activar los periféricos PWM.

Después, el programa ejecuta un ciclo infinito que atiende la comunicación serial, procesa las tareas del dimmer y recibe los comandos de la consola.

```
while (true) {
    tarea_uart();
    tarea_dimmer();
    consola_tarea();
}
```

Esta estructura permite mantener las tareas del sistema en ejecución continua y atender las operaciones solicitadas desde la consola.

Registro y procesamiento de datos

Para documentar las mediciones se utiliza el script graficar.py, encargado de procesar los datos capturados y generar archivos gráficos. El programa permite representar las muestras en función del tiempo y construir histogramas para el análisis de las mediciones.

Este procedimiento facilita la organización de los datos obtenidos durante las distintas actividades de la práctica.

Resultados

Análisis del ruido en las lecturas del ADC

Se realizaron capturas de 1000 muestras con una frecuencia de muestreo de 1000 Hz, manteniendo el potenciómetro en una posición fija. Se compararon las mediciones obtenidas con GP23 desactivado (**ps off**) y activado (**ps on**).

En la primera condición, las lecturas se concentraron principalmente en los códigos 2003 y 2004, con algunas variaciones hacia valores cercanos a 2005 y una lectura aislada de 2002. En la segunda condición, los valores se concentraron principalmente en los códigos 2004 y 2005, con algunas lecturas de 2003 y una lectura aislada de 2006.

Los histogramas muestran que la mayoría de las muestras se agrupan en pocos niveles digitales. Esto indica que, aunque el potenciómetro permaneció fijo, las lecturas presentaron pequeñas variaciones. La comparación permite observar diferencias entre ambas condiciones, aunque se requiere un análisis estadístico adicional para determinar con precisión la magnitud del ruido.

Muestreo de una señal PWM

Se analizaron las lecturas de una señal PWM de 1 kHz utilizando diferentes frecuencias de muestreo.

Con una frecuencia de muestreo de 10 000 Hz, la gráfica mostró transiciones periódicas entre un nivel bajo, cercano a cero, y un nivel alto, cercano a 4095. Esto permitió observar el comportamiento de la señal durante varios ciclos.

Al utilizar una frecuencia de muestreo de 1000 Hz, las lecturas permanecieron prácticamente constantes alrededor del código 4095. Este comportamiento se relaciona con la coincidencia entre la frecuencia de muestreo y la frecuencia de la señal, lo que provoca que las muestras puedan registrar repetidamente una misma parte del ciclo.

Posteriormente, con una frecuencia de muestreo de 900 Hz, se observaron variaciones entre distintos códigos digitales. La gráfica presentó fluctuaciones aproximadas entre 2087 y 2098, lo que muestra una representación diferente de la señal original debido a la relación entre ambas frecuencias.

En conjunto, las capturas muestran que la frecuencia de muestreo influye en la forma en que se representa una señal PWM y que una selección inadecuada puede producir aliasing.

Comparación de la respuesta de los filtros digitales

Se compararon las respuestas de tres condiciones de procesamiento: sin filtro, con media móvil y con media móvil exponencial (EMA). Para ello, se observó el comportamiento de las lecturas mientras se modificaba la posición del potenciómetro.

En la gráfica sin filtro, la transición entre el nivel bajo y el nivel alto se presentó aproximadamente entre 2200 y 2600 ms. Antes del cambio, las lecturas se mantuvieron cerca de cero y, después de la transición, se estabilizaron alrededor de 2800 códigos.

En la gráfica correspondiente a la media móvil, la transición comenzó aproximadamente entre 1700 y 2200 ms, alcanzando un nivel estable cercano a 3800 códigos. La curva mostró una transición gradual debido al promedio de las muestras utilizadas.

En la gráfica de EMA, el cambio comenzó aproximadamente entre 1450 y 2400 ms, hasta alcanzar un nivel cercano a 4080 códigos. La curva presentó una respuesta progresiva hacia el nuevo nivel.

Las gráficas muestran que las dos técnicas de filtrado generan transiciones suaves. Sin embargo, los niveles finales y los tiempos de transición son diferentes entre las

capturas, por lo que no deben atribuirse únicamente al filtro sin verificar que las condiciones iniciales y finales de las pruebas hayan sido iguales.

Registro de datos del dimmer en la consola

Durante las pruebas del dimmer se registraron mensajes en la consola con los campos level y cambios. El primero corresponde al nivel de control aplicado al PWM y el segundo indica el número de modificaciones detectadas durante el intervalo de medición.

En las capturas se observaron lecturas que permanecieron constantes durante varios ciclos, así como otras en las que el nivel cambió con mayor frecuencia. Al activar el comando dimmer y seleccionar el filtro EMA, se observaron secuencias en las que el nivel fue disminuyendo gradualmente.

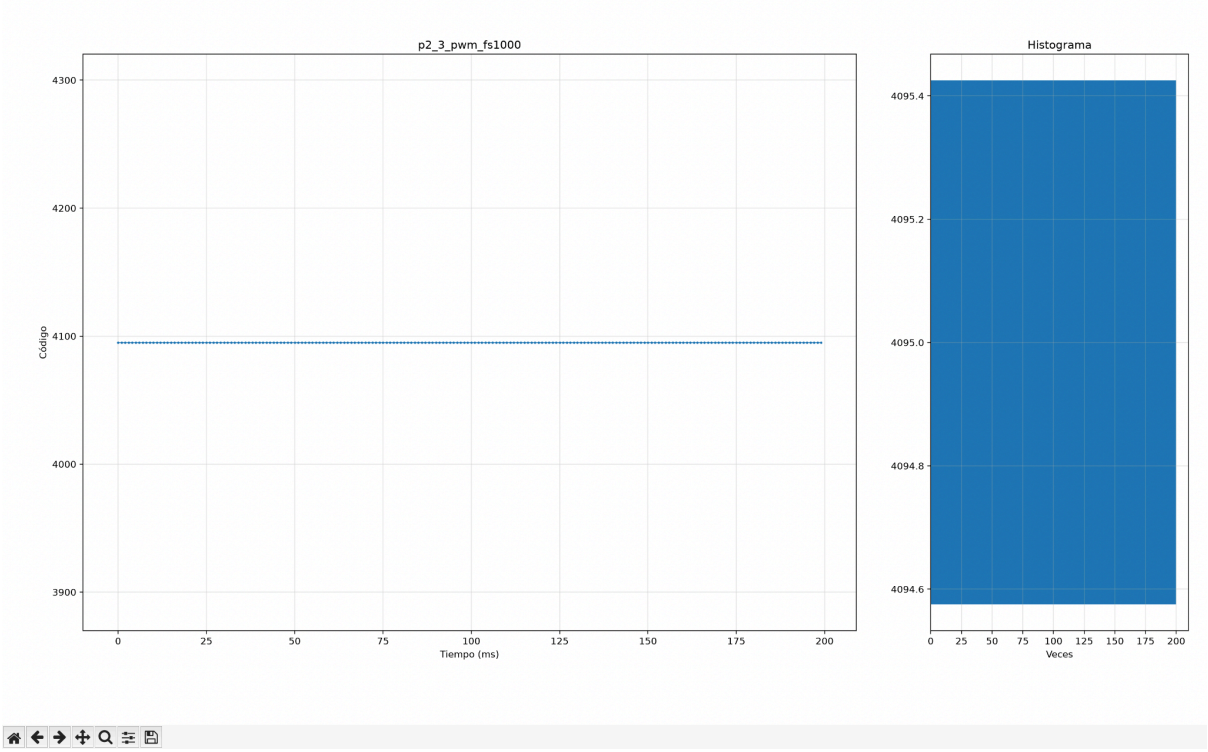
También se registraron periodos en los que el contador de cambios permaneció en cero, junto con otros en los que aumentó. Este comportamiento permite documentar las variaciones del nivel de control durante la ejecución del programa.

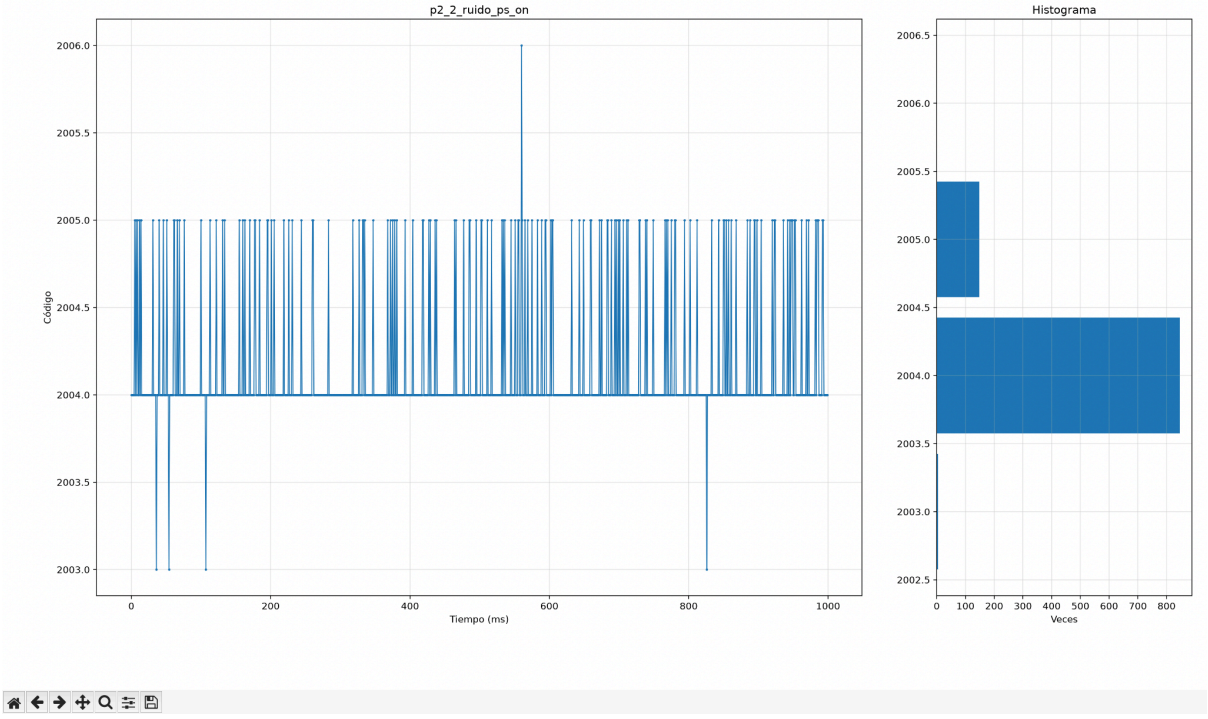
Los resultados obtenidos permitieron observar el comportamiento del ADC ante una entrada analógica prácticamente constante, identificar diferencias en las lecturas al modificar el estado de GP23 y comparar la representación de una señal PWM con distintas frecuencias de muestreo.

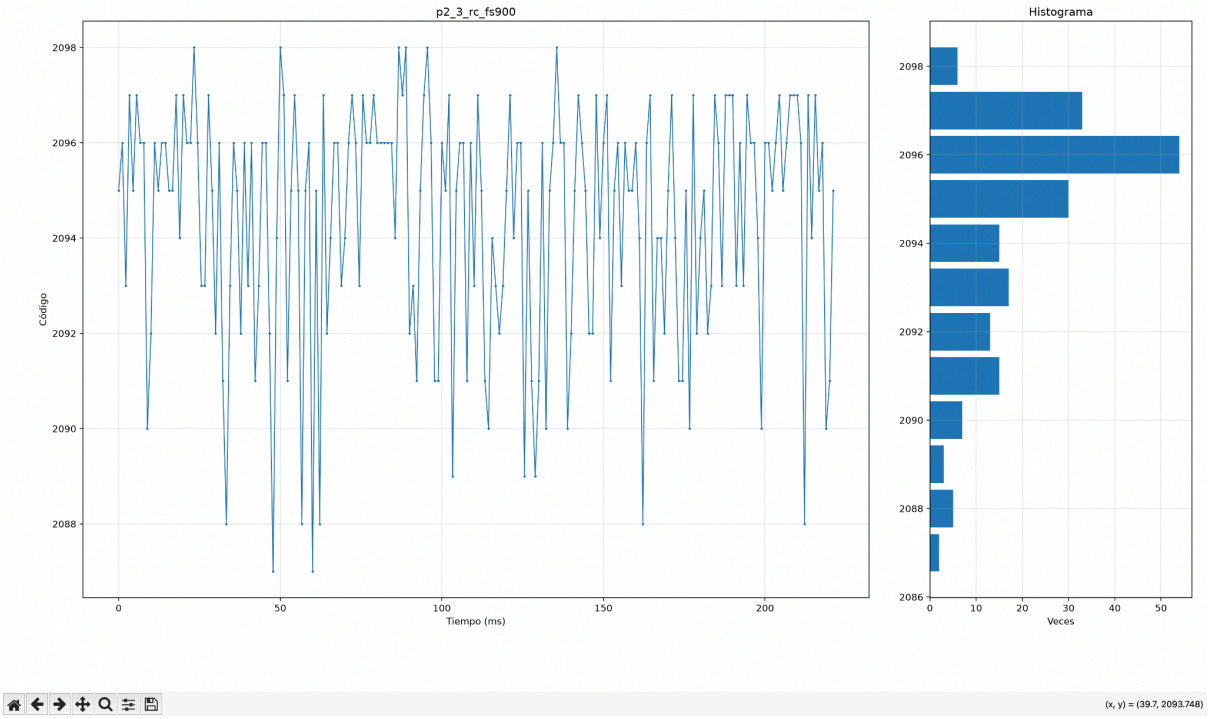
Asimismo, las gráficas permitieron comparar las transiciones producidas sin filtrado, con media móvil y con EMA. Finalmente, los registros de la consola documentaron las variaciones del nivel de control utilizado en las pruebas del dimmer.

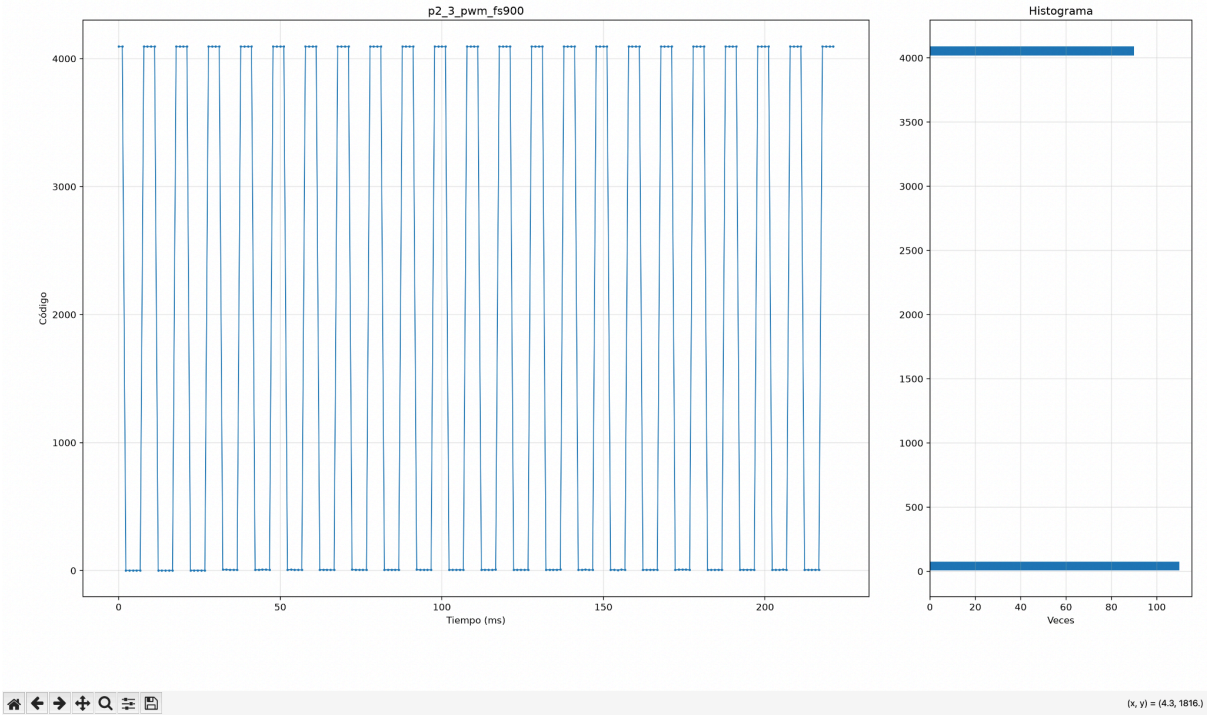
En conjunto, las mediciones y sus representaciones gráficas proporcionaron información para analizar el ruido, el muestreo de señales y el comportamiento de los métodos de filtrado digital.

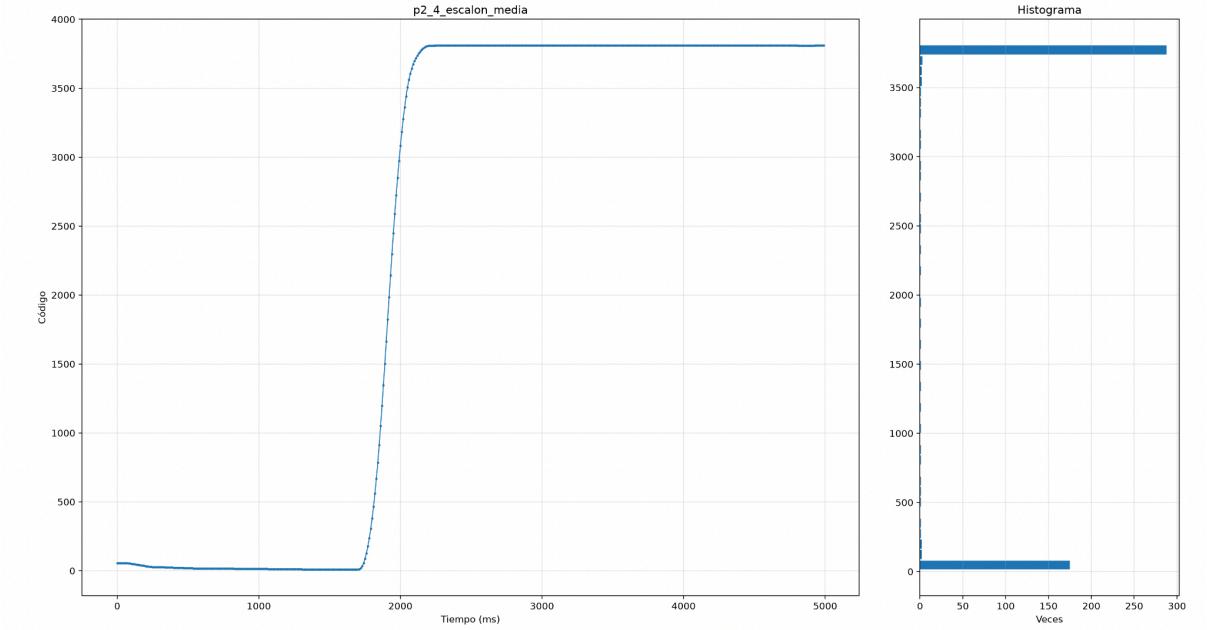
Evidencias

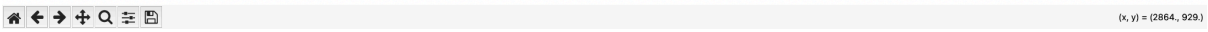

p2_3_pwm_fs1000
Código
Tiempo (ms)
Histograma
Veces

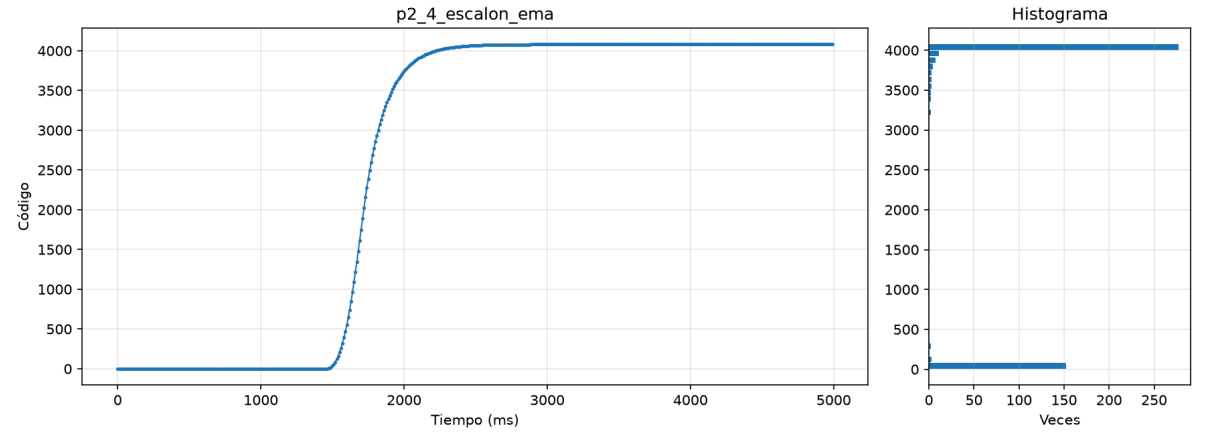

p2_2_ruido_ps_on
Código
Tiempo (ms)
Histograma
Veces

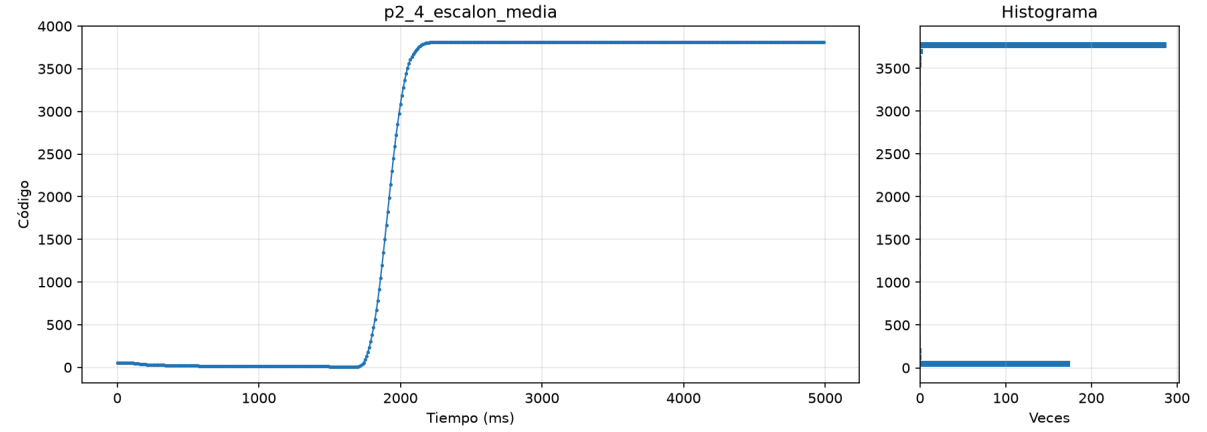

p2_3_rc_fs900
Código
Tiempo (ms)
Histograma
Veces
(x, y) = (39.7, 2093.748)

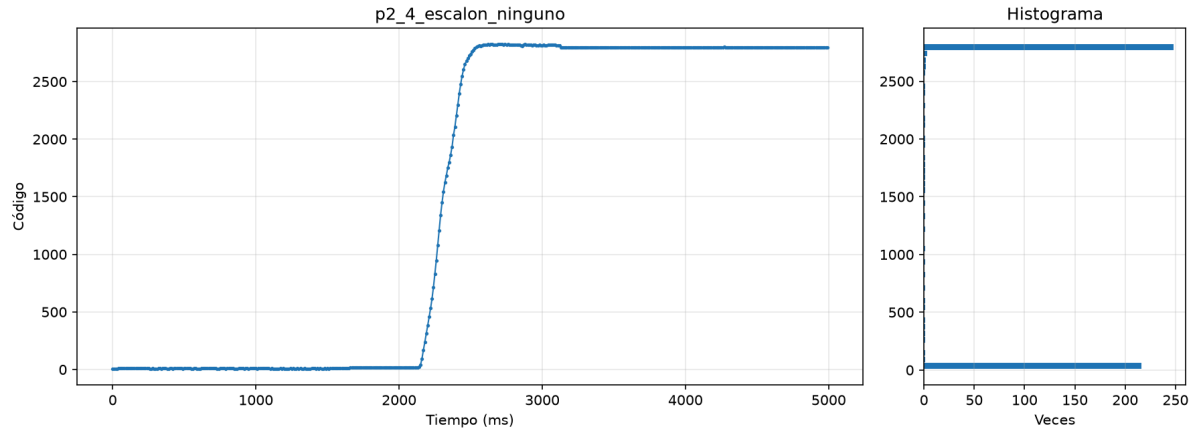

p2_3_pwm_fs900
Código
Tiempo (ms)
Histograma
Veces
(x, y) = (4.3, 1816.)


p2_4_escalon_media
Código
Tiempo (ms)
Histograma
Veces


(x, y) = (2864., 929.)

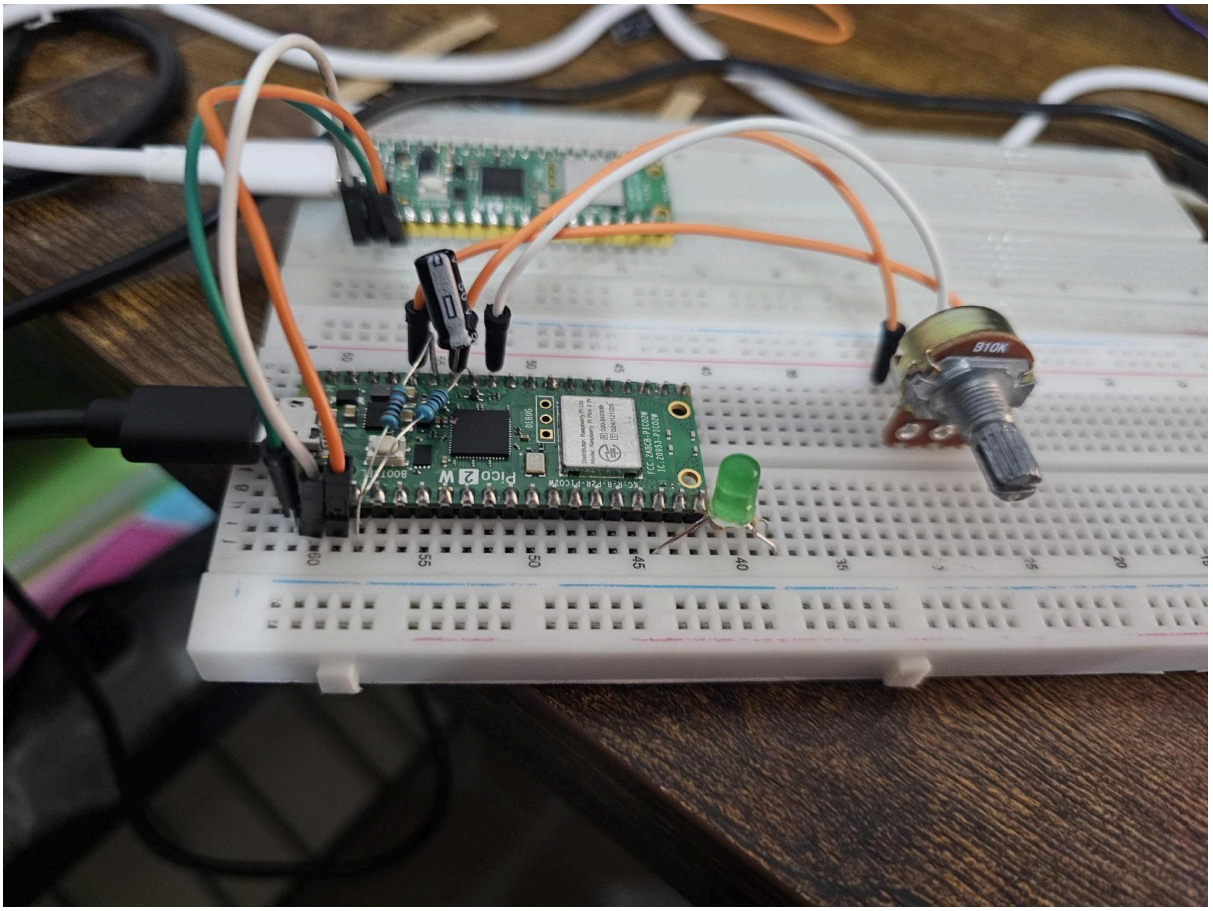

p2_4_escalon_ema
Código
Tiempo (ms)
Histograma
Veces


p2_4_escalon_media
Código
Tiempo (ms)
Histograma
Veces




```
> level = 429, cambios = 0
> level = 429, cambios = 0
> level = 429, cambios = 2
> level = 429, cambios = 4
> level = 429, cambios = 0
> level = 429, cambios = 0
> level = 429, cambios = 6
> level = 429, cambios = 2
> level = 429, cambios = 0
> level = 429, cambios = 2
> level = 429, cambios = 2
> level = 429, cambios = 0
> level = 429, cambios = 0
> level = 430, cambios = 5
> level = 430, cambios = 0
> level = 430, cambios = 0
> level = 429, cambios = 5
> level = 669, cambios = 42
> level = 900, cambios = 95
> level = 970, cambios = 65
> level = 938, cambios = 61
> level = 687, cambios = 97
> level = 533, cambios = 99
> level = 512, cambios = 37
> level = 525, cambios = 25
> level = 541, cambios = 30
> level = 595, cambios = 56
> level = 646, cambios = 52
> level = 535, cambios = 76
> level = 473, cambios = 61
> level = 403, cambios = 68
> level = 416, cambios = 49
> level = 537, cambios = 89
> level = 653, cambios = 78
> level = 714, cambios = 57
> level = 853, cambios = 90
> level = 996, cambios = 70
> level = 879, cambios = 38
> level = 619, cambios = 100
> level = 461, cambios = 98
> level = 449, cambios = 41
> level = 377, cambios = 60
> level = 314, cambios = 57
> level = 211, cambios = 83
> level = 122, cambios = 75
> level = 72, cambios = 64
> level = 107, cambios = 27
> level = 110, cambios = 5
> level = 122, cambios = 20
> level = 142, cambios = 23
> level = 154, cambios = 20
> level = 163, cambios = 13
> level = 187, cambios = 34
> level = 224, cambios = 44
> level = 256, cambios = 36
> level = 200, cambios = 49
> level = 172, cambios = 35
> level = 172, cambios = 0
>
--- exit ---
(.venv) anarojas@MacBook-Pro-de-Ana practica_adc_plantilla 2 %
```

```
> level = 43, cambios = 2
> level = 43, cambios = 2
> level = 43, cambios = 0
> level = 43, cambios = 0
> level = 43, cambios = 2
> level = 43, cambios = 2
> level = 43, cambios = 0
> dimmer off
error: comando desconocido
> level = 43, cambios = 2
> level = 43, cambios = 6
> level = 43, cambios = 0
> level = 43, cambios = 2
> dimmer offlevel = 43, cambios = 4
>
ok
> filtro ema
ok
> dimmer on
ok
> level = 936, cambios = 1
> level = 880, cambios = 1
> level = 828, cambios = 1
> level = 779, cambios = 1
> level = 733, cambios = 1
> level = 690, cambios = 1
> level = 649, cambios = 1
> level = 612, cambios = 1
> level = 576, cambios = 1
> level = 542, cambios = 1
> level = 511, cambios = 1
> level = 482, cambios = 1
> level = 455, cambios = 1
> level = 429, cambios = 1
> level = 405, cambios = 1
> level = 382, cambios = 1
> level = 361, cambios = 1
> level = 341, cambios = 1
> level = 322, cambios = 1
> level = 305, cambios = 1
> level = 289, cambios = 1
> level = 273, cambios = 1
> level = 259, cambios = 1
> level = 245, cambios = 1
> level = 233, cambios = 1
> level = 43, cambios = 54
> level = 43, cambios = 0
> level = 43, cambios = 0
> level = 43, cambios = 0
```



Conclusiones

En esta práctica se comprendió el funcionamiento del convertidor analógico-digital de la Raspberry Pi Pico 2 y su aplicación en la lectura de señales analógicas. Mediante las diferentes pruebas se estudió cómo influyen la frecuencia de muestreo y el ruido en las mediciones, así como la forma en que el aliasing puede provocar una representación incorrecta de una señal PWM.

También se revisó la aplicación de filtros digitales, como la media móvil y la media móvil exponencial (EMA), para suavizar las variaciones de las lecturas. Finalmente, se relacionó la adquisición de datos analógicos con el control del brillo de un LED mediante PWM, integrando conceptos de conversión, procesamiento digital y control de señales.

En general, la práctica permitió relacionar los conceptos teóricos con el funcionamiento de un sistema embebido y reconocer la importancia de seleccionar adecuadamente la frecuencia de muestreo y los métodos de procesamiento para obtener mediciones más estables y confiables.

REFERENCIAS

Analog Devices. (s. f.). *MT-002: What the Nyquist criterion means to your sampled data system design*. https://www.analog.com/media/en/training-seminars/tutorials/MT-002.pdf

Microchip Technology Inc. (s. f.). *Analog-to-digital converter (ADC)*. https://www.microchip.com/

OpenAI. (2026). *ChatGPT* [Modelo de lenguaje grande]. https://chatgpt.com/

Raspberry Pi Ltd. (s. f.). *Hardware APIs: Analog-to-digital converter (ADC)*. Raspberry Pi Documentation. https://www.raspberrypi.com/documentation/pico-sdk/hardware.html

Raspberry Pi Ltd. (s. f.). *Raspberry Pi Pico-series microcontrollers*. Raspberry Pi Documentation. https://www.raspberrypi.com/documentation/microcontrollers/pico-series.html

Raspberry Pi Ltd. (s. f.). *RP2350 datasheet*. Raspberry Pi Documentation. https://pip.raspberrypi.com/categories/1214-rp2350